



Department of Computer Science and Engineering (Data Science)

Experiment-9

(Outlier Detection)

Aim: Perform different outlier detection methods on given data.

Theory:

Anomaly detection (aka outlier analysis) is a step in data mining that identifies data points, events, and/or observations that deviate from a dataset's normal behavior. Anomalous data can indicate critical incidents, such as a technical glitch, or potential opportunities, for instance a change in consumer behavior.

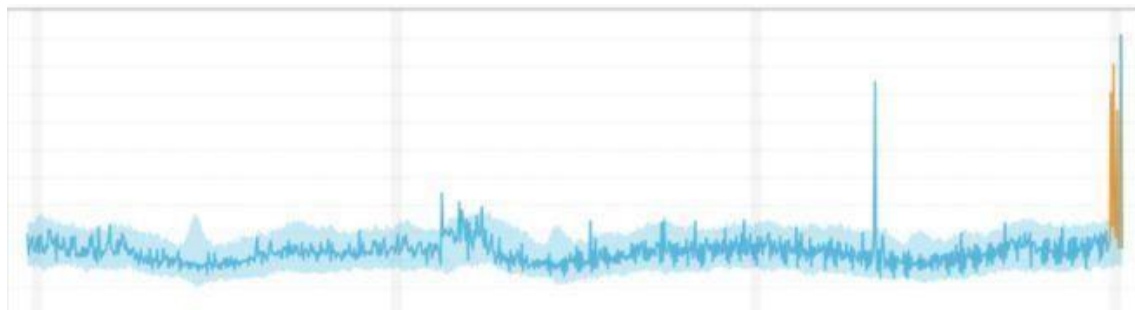
With all the analytics programs and various management software available, it's now easier than ever for companies to effectively measure every single aspect of business activity. This includes the operational performance of applications and infrastructure components as well as key performance indicators (KPIs) that evaluate the success of the organization. With millions of metrics that can be measured, companies tend to end up with quite an impressive dataset to explore the performance of their business.

Within this dataset are data patterns that represent business as usual. An unexpected change within these data patterns, or an event that does not conform to the expected data pattern, is considered an anomaly. In other words, an anomaly is a deviation from business as usual.

Generally speaking, anomalies in your business data fall into [three main categories](#) — global outliers, contextual outliers, and collective outliers.

1. Global outliers

Also known as point anomalies, these outliers exist far outside the entirety of a data set.



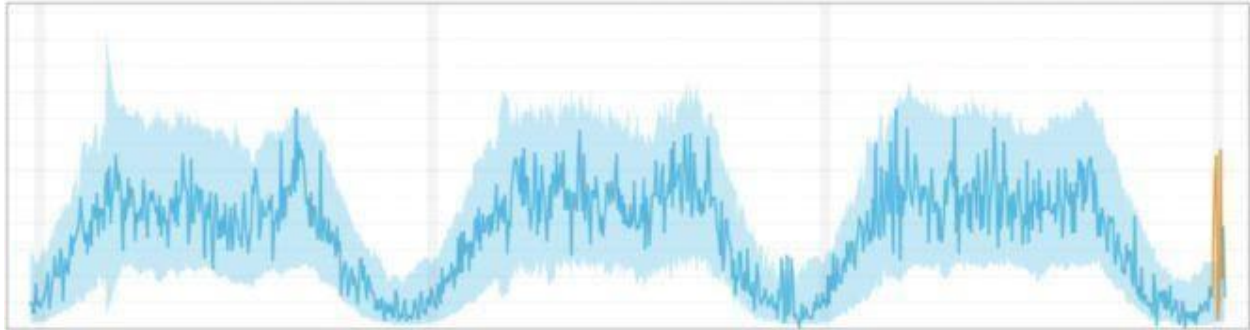
2. Contextual outliers

Also called conditional outliers, these anomalies have values that significantly deviate from the other data points that exist in the same context. An anomaly in the context of one dataset may not be an anomaly in another. These outliers are common in time series data because those datasets are records of specific quantities in a given period. The value exists within global expectations



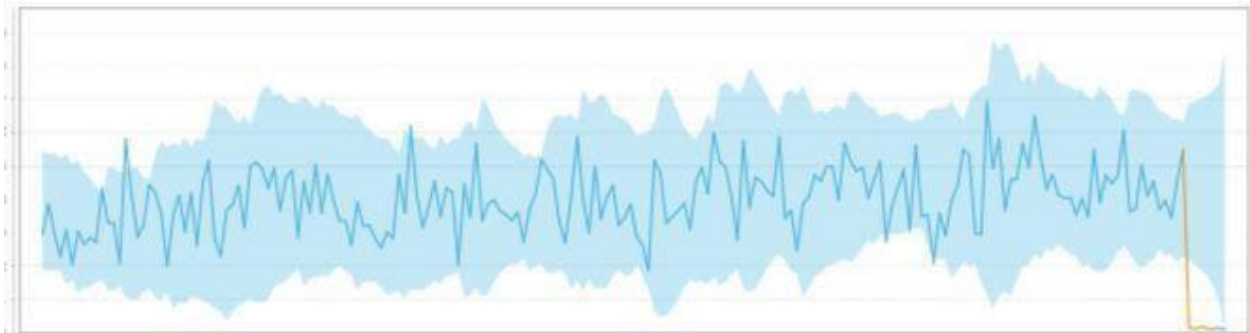
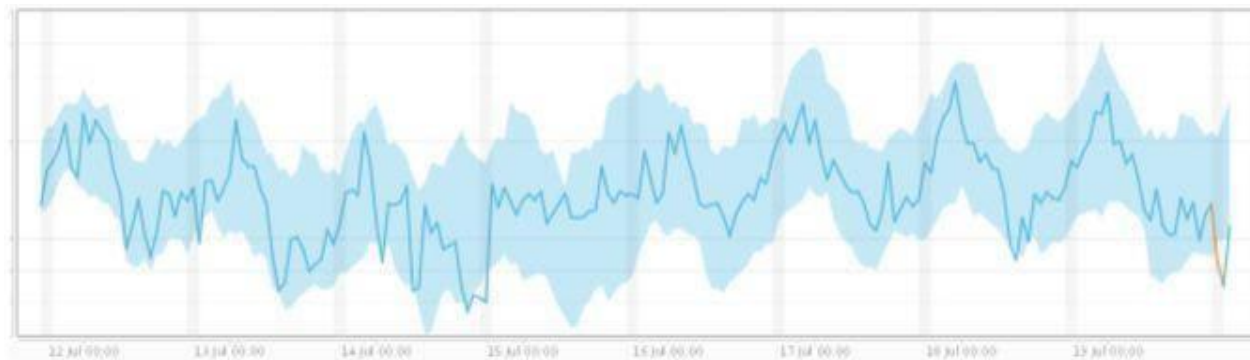
Department of Computer Science and Engineering (Data Science)

but may appear anomalous within certain seasonal data patterns.



3. Collective outliers

When a subset of data points within a set is anomalous to the entire dataset, those values are called collective outliers. In this category, individual values aren't anomalous globally or contextually. You start to see these types of outliers when examining distinct time series together. Individual behavior may not deviate from the normal range in a specific time series dataset. But when combined with another time series dataset, more significant anomalies become clear.



Z-score

Simply speaking, Z-score is a statistical measure that tells you how far is a data point from the rest of the



Department of Computer Science and Engineering (Data Science)

dataset. In a more technical term, Z-score tells how many standard deviations away a given observation is from the mean. For example, a Z score of 2.5 means that the data point is 2.5 standard deviation far from the mean. And since it is far from the center, it's flagged as an outlier/anomaly.

Z-score is a parametric measure and it takes two parameters — mean and standard deviation.

Once you calculate these two parameters, finding the Z-score of a data point is easy.

$$Z\text{-score} = \frac{x - \text{mean}}{\text{Standard Deviation}}$$

Note that mean and standard deviation are calculated for the whole dataset, whereas x represents every single data point. That means, every data point will have its own z-score, whereas mean/standard deviation remains the same everywhere.

kNN for anomaly detection

Although kNN is a supervised ML algorithm, when it comes to anomaly detection it takes an unsupervised approach. This is because there is no actual “learning” involved in the process and there is no pre-determined labeling of “outlier” or “not-outlier” in the dataset, instead, it is entirely based upon threshold values. Data scientists arbitrarily decide the cutoff values beyond which all observations are called anomalies (as we will see later). That is also why there is no train-test-split of data or an accuracy report.



Local Outlier Factor (LOF)

LOF is an unsupervised (well, semi-supervised) machine learning algorithm that uses the density of data points in the distribution as a key factor to detect outliers.

LOF compares the density of any given data point to the density of its neighbors. Since outliers come from low-density areas, the ratio will be higher for anomalous data points. As a rule of thumb, a normal data point has a LOF between 1 and 1.5 whereas anomalous observations will have much higher LOF. The

**Department of Computer Science and Engineering (Data Science)**

higher the LOF the more likely it is an outlier. If the LOF of point X is 5, it means the average density of X's neighbors is 5 times higher than its local density.

In mathematical terms,

$LOF(X) = [(LRD(1st\ neighbor) + LRD(2nd\ neighbor) + \dots + LRD(kth\ neighbor)) / LRD(X)] / k$

where LRD is Local Reachability Distance and is computed as follows.

$LRD(X) = 1 / (\text{sum of Reachability Distance}(X, n) / k)$ where n is neighbors upto k

Lab Assignments to complete in this session

1. For SAT dataset show the outliers using z-score and modifier z-score.
2. For Football data show the outliers using z-score and modifier z-score.
3. Generate 8 random points and perform KNN and LOF for K=1 and K=3. Write your observations.
- 4.

A. Create a function `do_nn_avg_scores(obs, n_neighbors=1)` that computes outlier scores using arithmetic mean distance from the point to each of the `n_neighbors` nearest neighbors as the score.

B. Do the same thing as in part (A) to create `do_nn_harm_scores(obs, n_neighbors=1)`, where you use the harmonic mean instead of the mean. The harmonic mean of `n` points is defined as

$$\begin{aligned} \text{harmonic}(X_1, X_2, \dots, X_n) &= \frac{n}{(1/x_1) + (1/x_2) + \dots + (1/x_n)} = \frac{(\prod X_i)^{1/n}}{\bar{X}} \\ &= \frac{(X_1 X_2 \cdot X_n)^{1/n}}{\bar{X}} \end{aligned}$$

Note that `scipy.stats` contains a `hmean` function you can use.

5. This exercise refers to density-based methods (Local Outlier Factor; Section 3).
 - A. Create a function `do_lof_outlier_scores(obs, n_neighbors=3)` that computes outlier scores using the LOF method. Recall that the values returned by sklearn's implementation are negatives of what we want.

FDA Experiment-9

1.

```
0s [49] %matplotlib inline

import sys
import scipy
import scipy.stats as ss
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
import pandas as pd
import random
```

```
0s [49] ct_test = pd.read_csv('/content/SAT_CT_District_Participation_2012.csv')
print(ct_test.shape)
ct_test.head()
```

(130, 2)

	District	Participation Rate
0	Ansonia	0.66
1	Avon	0.90
2	Berlin	0.81
3	Bethel	0.86
4	Bloomfield	0.81

```
0s [51] print('Mean participation rate is {:.3f}'.format(mean_rate))
print('Standard deviation is {:.3f}'.format(stdev_rate))
```

Mean participation rate is 0.741
Standard deviation is 0.136

Calculate the z-score and add the result to the dataframe.

```
0s [52] zscore_rate = ss.zscore(ct_test['Participation Rate'], ddof=0)
ct_test = ct_test.assign(zscore=zscore_rate)
ct_test.head(8)
```

	District	Participation Rate	zscore
0	Ansonia	0.66	-0.593779
1	Avon	0.90	1.175649
2	Berlin	0.81	0.512113
3	Bethel	0.86	0.880744
4	Bloomfield	0.81	0.512113
5	Bolton	0.85	0.807018
6	Branford	0.77	0.217209
7	Bridgeport	0.58	-1.183589

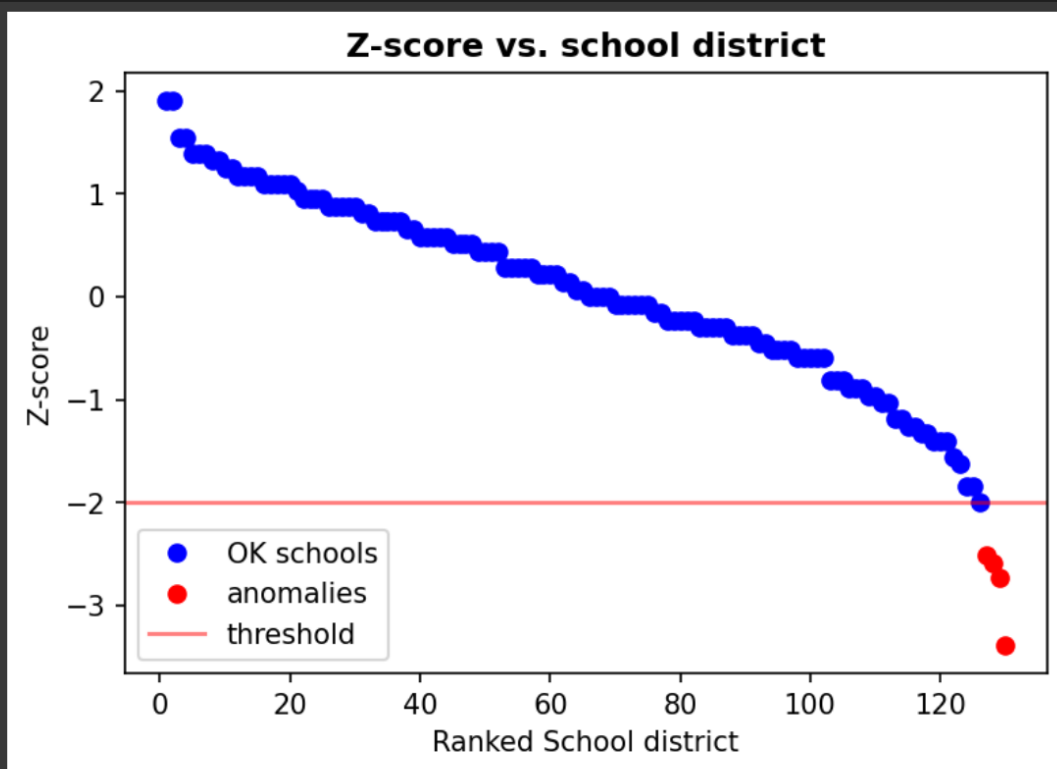
```

def plot_anomaly(score_data, threshold):
    # Mask to plot values above and below threshold in different colors
    score_data = score_data.copy().sort_values(ascending=False).values
    ranks = np.linspace(1, len(score_data), len(score_data))
    mask_outlier = (score_data < threshold)

    plt.figure(dpi=150)
    plt.plot(ranks[~mask_outlier], score_data[~mask_outlier], 'o', color='b', label='OK schools')
    plt.plot(ranks[mask_outlier], score_data[mask_outlier], 'o', color='r', label='anomalies')
    plt.axhline(threshold, color='r', label='threshold', alpha=0.5)
    plt.legend(loc = 'lower left')
    plt.title('Z-score vs. school district', fontweight='bold')
    plt.xlabel('Ranked School district')
    plt.ylabel('Z-score')
    plt.show()

```

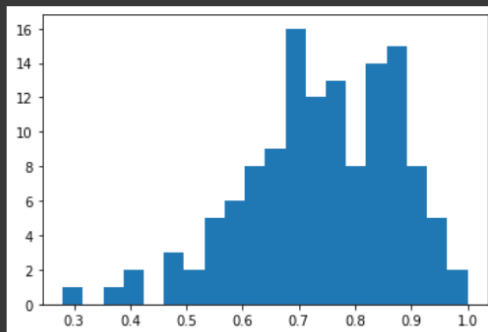
```
[54] plot_anomaly(ct_test['zscore'], -2)
```



```
[55] zscore_anomalies = ct_test[(ct_test['zscore'] < -2)]
zscore_anomalies
```

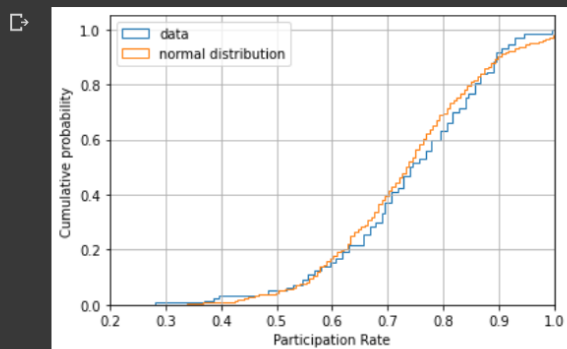
	District	Participation Rate	zscore
50	New Britain	0.40	-2.510660
99	Windham	0.37	-2.731839
121	Eastern Connecticut Regional Educational Servi...	0.39	-2.584386
125	Stamford Academy	0.28	-3.395374

```
[56] nbins= 20
n_hist, bins_hist, patches_hist = plt.hist(ct_test['Participation Rate'], nbins, density=False,
cumulative=False, linewidth=1.0, label='data')
```



```
num_bins = 130
normal_dist = [random.gauss(mean_rate, stdev_rate) for _ in range(500)]
n, bins, patches = plt.hist(ct_test['Participation Rate'], num_bins, density=True, histtype='step',
cumulative=True, linewidth=1.0, label='data')
plt.hist(normal_dist, num_bins, density=True, histtype='step',
cumulative=True, linewidth=1.0, label='normal distribution')

plt.grid(True)
plt.legend(loc='upper left')
axes = plt.gca()
axes.set_xlim([0.2,1.0])
plt.xlabel('Participation Rate')
plt.ylabel('Cumulative probability')
plt.show()
```



+ Code + Text

```
[58] median_goals = np.median(ct_test['Participation Rate'])
median_goals
```

0.745

```
[59] def modified_zscore(data, consistency_correction=1.4826):
    """
    Returns the modified z score and Median Absolute Deviation (MAD) from the scores in data.
    The consistency_correction factor converts the MAD to the standard deviation for a given
    distribution. The default value (1.4826) is the conversion factor if the underlying data
    is normally distributed
    """
    median = np.median(data)

    deviation_from_med = np.array(data) - median

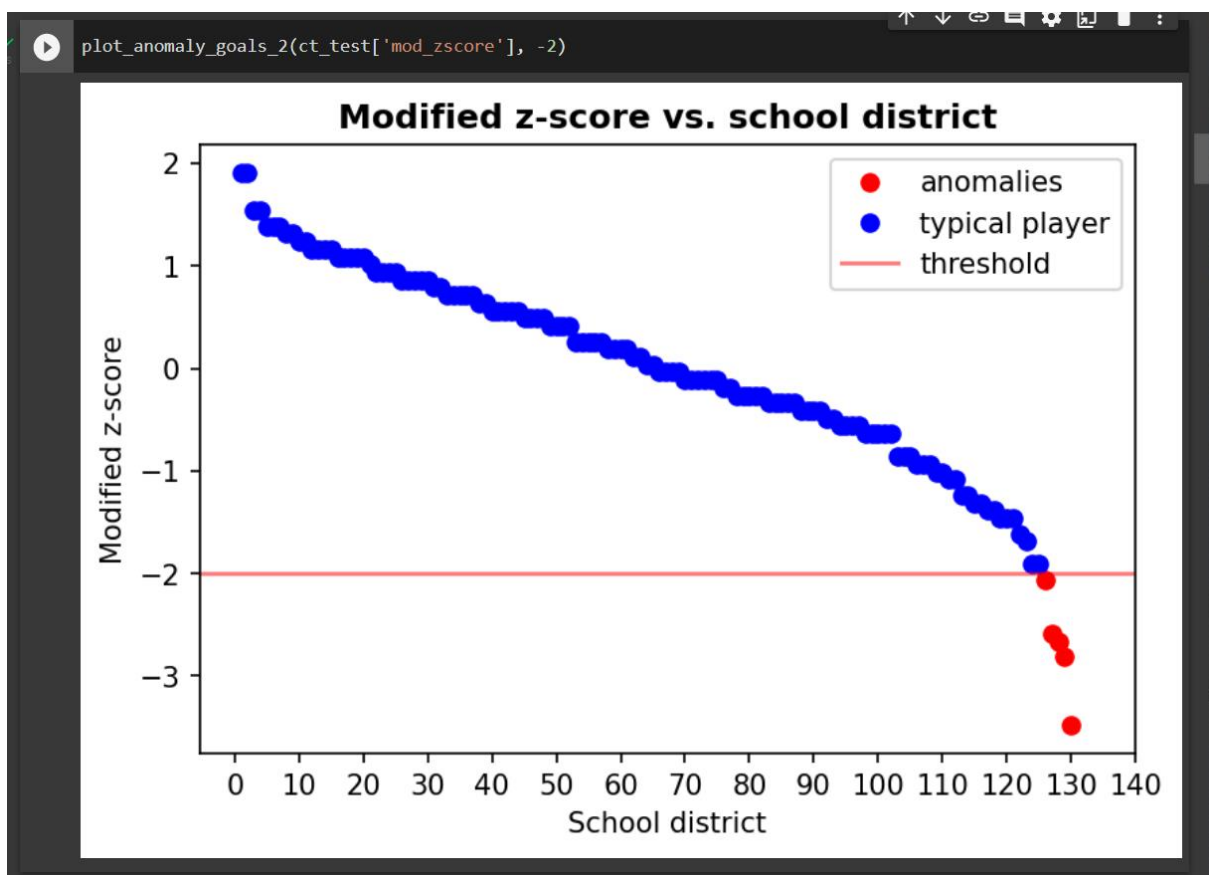
    mad = np.median(np.abs(deviation_from_med))
    mod_zscore = deviation_from_med/(consistency_correction*mad)
    return mod_zscore, mad
```



```
[102] mod_zscore_goals, mad_goals = modified_zscore(ct_test['Participation Rate'])
      ct_test = ct_test.assign(mod_zscore=mod_zscore_goals)

[132] def plot_anomaly_goals_2(score_data, threshold):
      score_data = score_data.copy().sort_values(ascending=False).values
      ranks = np.linspace(1, len(score_data), len(score_data))
      mask_outliers = (score_data < threshold)

      plt.figure(dpi=150)
      plt.plot(ranks[mask_outliers], score_data[mask_outliers], 'o', color='r', label='anomalies')
      plt.plot(ranks[~mask_outliers], score_data[~mask_outliers], 'o', color='b', label='typical player')
      plt.axhline(threshold, color='r', label='threshold', alpha=0.5)
      plt.legend(loc = 'upper right')
      plt.title('Modified z-score vs. school district', fontweight='bold')
      plt.xticks(np.arange(0, 150, step=10))
      plt.xlabel('School district')
      plt.ylabel('Modified z-score')
      plt.show()
```



```
[115] mod_zscore_anomalies_players = ct_test[(ct_test['mod_zscore'] < -2)]
      mod_zscore_anomalies_players
```

	District	Participation Rate	zscore	mod_zscore
50	New Britain	0.40	-2.510660	-2.585548
55	New London	0.47	-1.994577	-2.060944
99	Windham	0.37	-2.731839	-2.810378
121	Eastern Connecticut Regional Educational Servi...	0.39	-2.584386	-2.660491
125	Stamford Academy	0.28	-3.395374	-3.484869

2.

▼ Z-Score vs. Modified Z-Score—World Cup Top Goal Scorers

We will now look at a dataset that shows the limitations of z-scores and why the modified z-score can be useful. We will look at the number of goals scored by the top goalscorer in every World Cup from 1930 through 2018 (21 competitions in total). The raw data is from here: https://en.wikipedia.org/wiki/FIFA_World_Cup_top_goalscorers The data was cleaned and saved as a CSV. Load it as a dataframe and take a look.

```
# First row is a header row, but the header names are cumbersome.
# Skip first row and manually label
# Encoding ensures that accents in names are rendered properly
top_goals = pd.read_csv('/content/world_cup_top_goal_scorers.csv',
                        encoding='utf-8',
                        names=['Year', 'Player(s)', 'Goals'], skiprows=1)
top_goals.head()
```

	Year	Player(s)	Goals
0	1930	Guillermo Stábile	8
1	1934	Oldřich Nejedlý	5
2	1938	Leônidas	7
3	1950	Ademir	8
4	1954	Sándor Kocsis	11

```
mean_goals = top_goals['Goals'].mean()
stdev_goals = top_goals['Goals'].std(ddof=0)
print('Mean number of goals is {:.2f}'.format(mean_goals))
print('Standard deviation is {:.2f}'.format(stdev_goals))
```

Mean number of goals is 7.05
Standard deviation is 2.15

```

[67] zscore_goals = ss.zscore(top_goals['Goals'], ddof=0)
top_goals = top_goals.assign(zscore=zscore_goals)
top_goals.head(21)

```

	Year	Player(s)	Goals	zscore
0	1930	Guillermo Stábile	8	0.443242
1	1934	Oldřich Nejedlý	5	-0.952971
2	1938	Leônidas	7	-0.022162
3	1950	Ademir	8	0.443242
4	1954	Sándor Kocsis	11	1.839455
5	1958	Just Fontaine	13	2.770264
6	1962	Garrincha, Vavá, Leonel Sánchez, Flórián Alber...	4	-1.418375
7	1966	Eusébio	9	0.908647
8	1970	Gerd Müller	10	1.374051
9	1974	Grzegorz Lato	7	-0.022162
10	1978	Mario Kempes	6	-0.487566
11	1982	Paolo Rossi	6	-0.487566
12	1986	Gary Lineker	6	-0.487566
13	1990	Salvatore Schillaci	6	-0.487566
14	1994	Hristo Stoichkov, Oleg Salenko	6	-0.487566
15	1998	Davor Šuker	6	-0.487566
16	2002	Ronaldo	8	0.443242
17	2006	Miroslav Klose	5	-0.952971
18	2010	Thomas Müller, David Villa, Wesley Sneijder, D...	5	-0.952971
19	2014	James Rodríguez	6	-0.487566
20	2018	Harry Kane	6	-0.487566

+ Code + Text

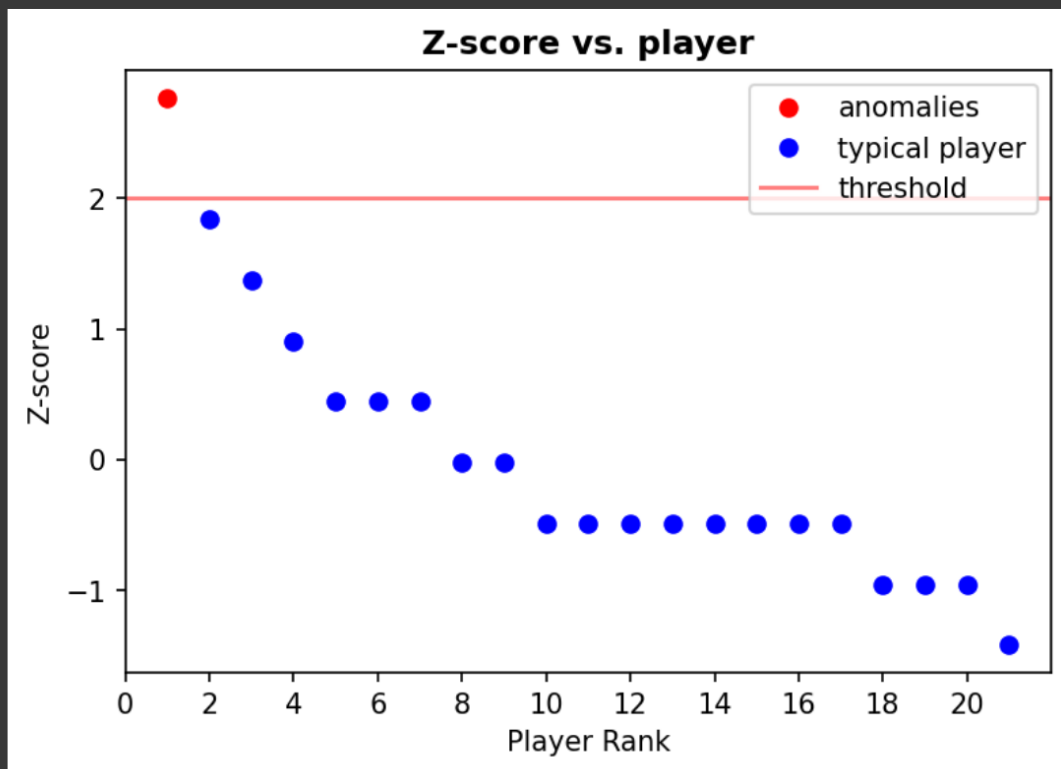
```

[68] def plot_anomaly_goals(score_data, threshold):
    score_data = score_data.copy().sort_values(ascending=False).values
    ranks = np.linspace(1, len(score_data), len(score_data))
    mask_outlier = (score_data > threshold)

    plt.figure(dpi=150)
    plt.plot(ranks[mask_outlier], score_data[mask_outlier], 'o', color='r', label='anomalies')
    plt.plot(ranks[~mask_outlier], score_data[~mask_outlier], 'o', color='b', label='typical player')
    plt.axhline(threshold, color='r', label='threshold', alpha=0.5)
    plt.legend(loc='upper right')
    plt.title('Z-score vs. player', fontweight='bold')
    plt.xticks(np.arange(0, 21, step=2.0))
    plt.xlabel('Player Rank')
    plt.ylabel('Z-score')
    plt.show()

```

```
[69] plot_anomaly_goals(top_goals['zscore'], 2)
```



```
median_goals = np.median(top_goals['Goals'])
median_goals
```

6.0

```
[72] def modified_zscore(data, consistency_correction=1.4826):
    """
    Returns the modified z score and Median Absolute Deviation (MAD) from the scores in data.
    The consistency_correction factor converts the MAD to the standard deviation for a given
    distribution. The default value (1.4826) is the conversion factor if the underlying data
    is normally distributed
    """
    median = np.median(data)

    deviation_from_med = np.array(data) - median

    mad = np.median(np.abs(deviation_from_med))
    mod_zscore = deviation_from_med/(consistency_correction*mad)
    return mod_zscore, mad
```

```

[73] mod_zscore_goals, mad_goals = modified_zscore(top_goals['Goals'])
      top_goals = top_goals.assign(mod_zscore=mod_zscore_goals)

[74] def plot_anomaly_goals_2(score_data, threshold):
      score_data = score_data.copy().sort_values(ascending=False).values
      ranks = np.linspace(1, len(score_data), len(score_data))
      mask_outliers = (score_data > threshold)

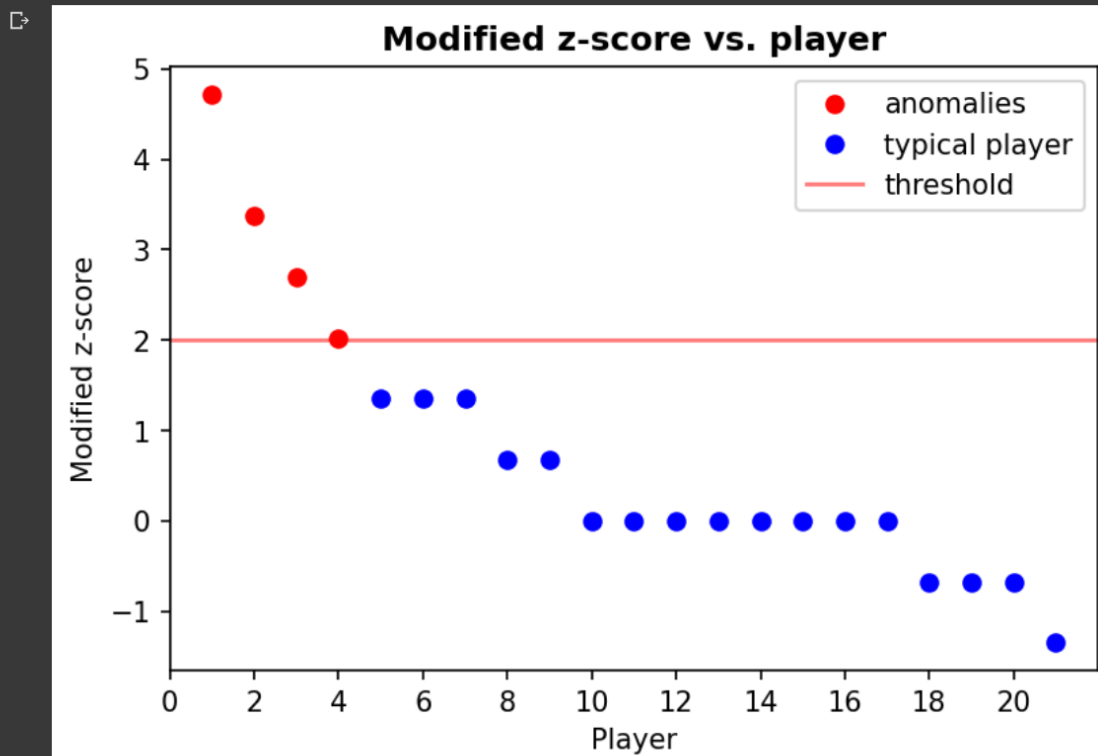
      plt.figure(dpi=150)
      plt.plot(ranks[mask_outliers], score_data[mask_outliers], 'o', color='r', label='anomalies')
      plt.plot(ranks[~mask_outliers], score_data[~mask_outliers], 'o', color='b', label='typical player')
      plt.axhline(threshold, color='r', label='threshold', alpha=0.5)
      plt.legend(loc = 'upper right')
      plt.title('Modified z-score vs. player', fontweight='bold')
      plt.xticks(np.arange(0, 21, step=2.0))
      plt.xlabel('Player')
      plt.ylabel('Modified z-score')
      plt.show()

```

```

plot_anomaly_goals_2(top_goals['mod_zscore'], 2)

```

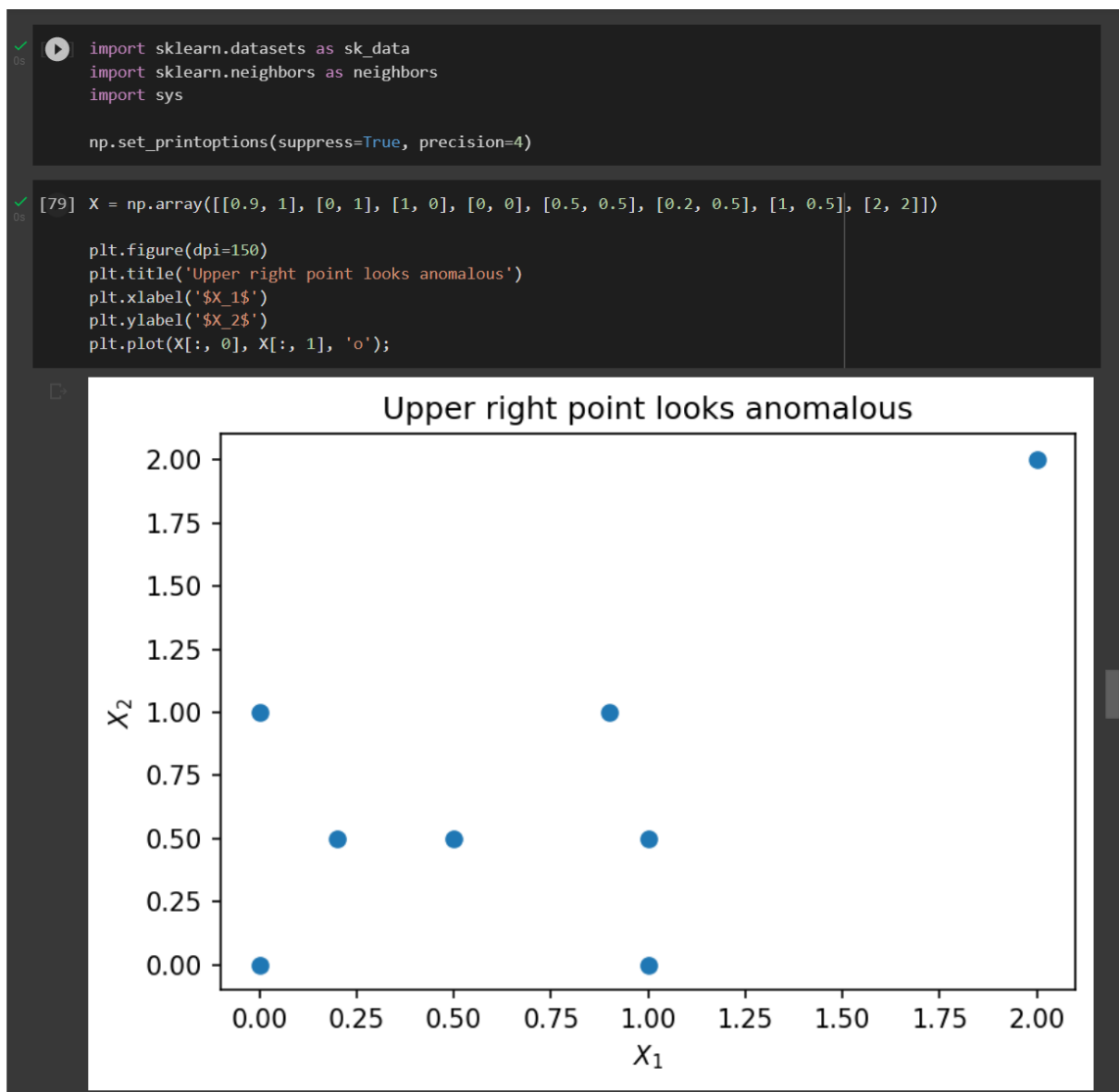


```
0s [76] mod_zscore_anomalies_players = top_goals[(top_goals['mod_zscore'] > 2)]
mod_zscore_anomalies_players
```

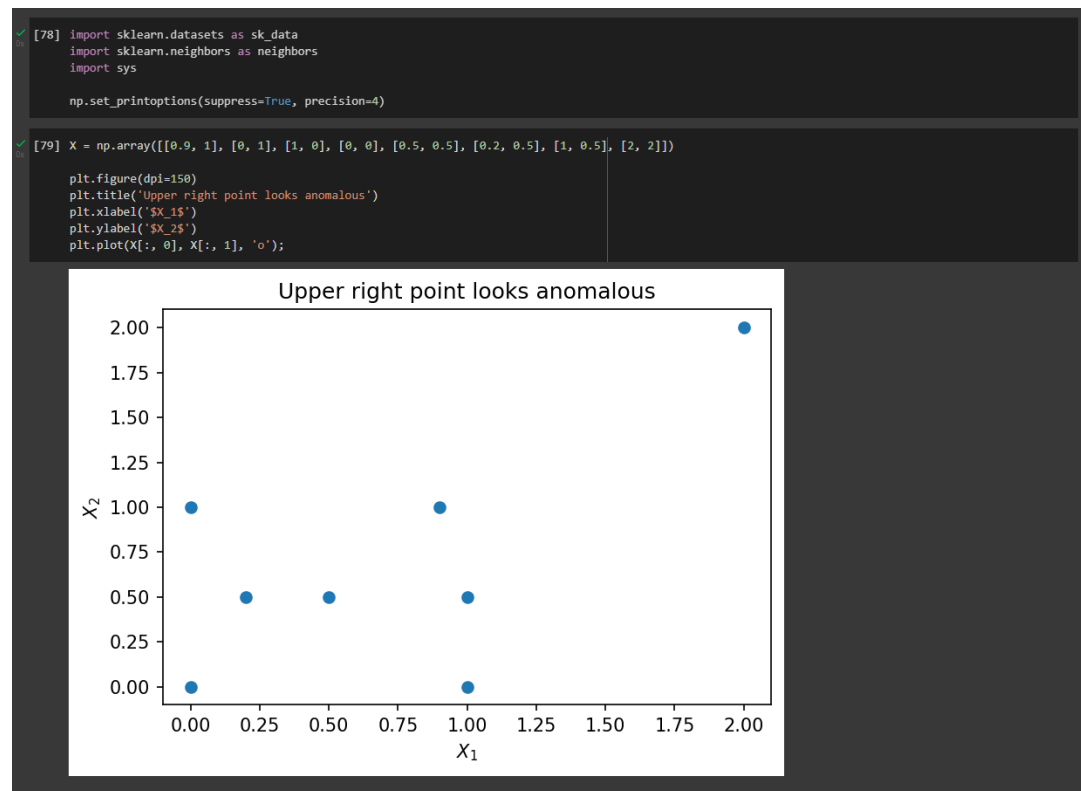
	Year	Player(s)	Goals	zscore	mod_zscore
4	1954	Sándor Kocsis	11	1.839455	3.372454
5	1958	Just Fontaine	13	2.770264	4.721435
7	1966	Eusébio	9	0.908647	2.023472
8	1970	Gerd Müller	10	1.374051	2.697963

```
0s [77] print('The value of MAD is {:.2f}'.format(mad_goals))
```

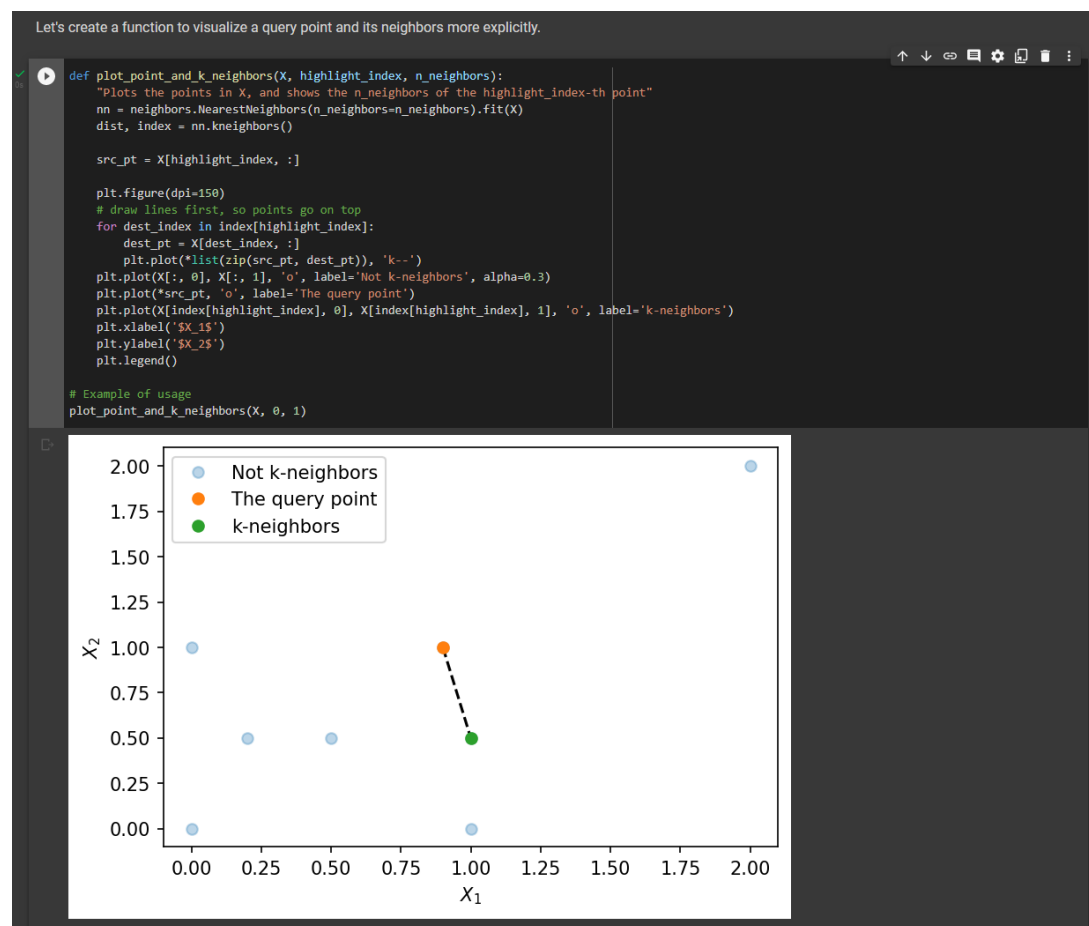
The value of MAD is 1.00

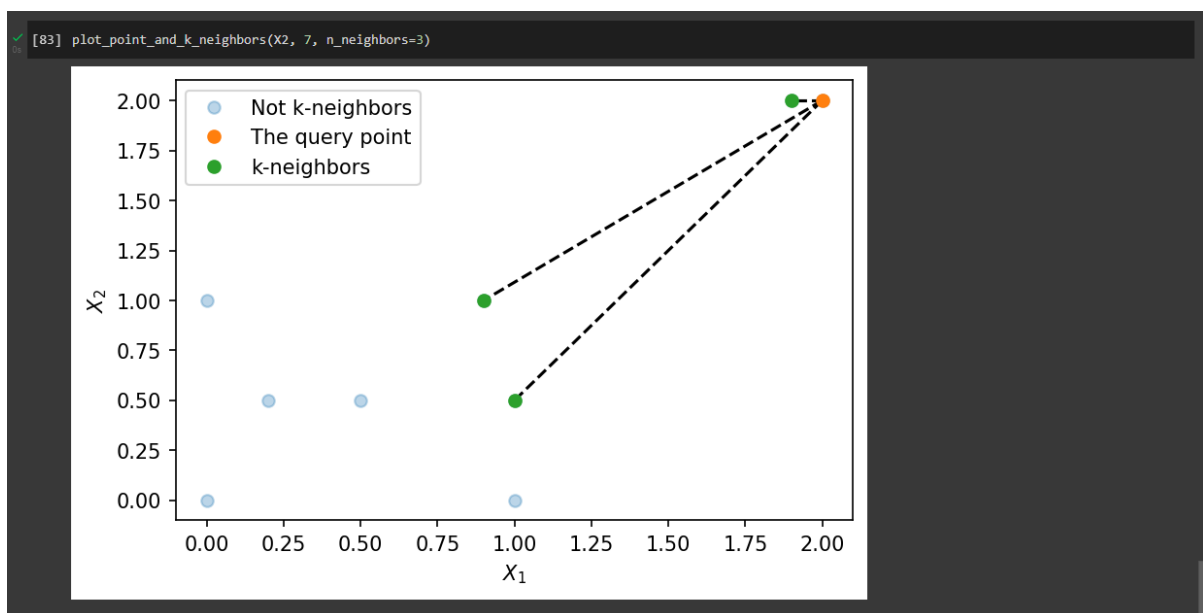


3.



KNN





Observation 1: dependence on k

The distance between point 7 and the new point is actually quite small—smaller than the distances between any pair of the points in the lower left. If $k=1$, the two points in the upper right would be the last ones to be classified as an anomaly (i.e. they would have the lowest score).

Observation 2: how to weight distances?

Let's look at the same data set (i.e. with the additional point) but now look at $k=3$

We see one of the 3 nearest neighbors is very close, but the other two are far away. We need some way of combining these 3 distances into a single score. The three commonly used methods are Use the longest distance Use the (arithmetic) mean distance Use the harmonic mean. We will implement the longest distance, and assign the arithmetic mean distance and harmonic mean distance as exercises.

LOF:

▼ Density-Based Methods (Local Outlier Factor)

The local outlier factor (LOF) is sklearn has a version available for us to use. Note the the LOF returned by sklearn is the negative of the value we defined in the lecture. Apply LOF to the collection of points ('X2') we created in section 1.

```
import sklearn.neighbors as neighbors
lof = neighbors.LocalOutlierFactor(n_neighbors=1, contamination='auto')
lof.fit(X2)
sk_lof = -lof.negative_outlier_factor_
print_ranked_scores(X2, sk_lof)
```

Rank	Point	Score
1.	[2. 2.]	1.8608
2.	[1.9 2.]	1.8293
3.	[0.2 0.5]	1.2154
4.	[1. 0.5]	1.2091
5.	[0. 0.]	1.0333
6.	[0. 1.]	1.0120
7.	[1. 0.]	0.9554
8.	[0.9 1.]	0.8891
9.	[0.5 0.5]	0.8272

▼ Density-Based Methods (Local Outlier Factor)

The local outlier factor (LOF) is sklearn has a version available for us to use. Note the the LOF returned by sklearn is the negative of the value we defined in the lecture. Apply LOF to the collection of points (X2) we created in section 1.

```
import sklearn.neighbors as neighbors
lof = neighbors.LocalOutlierFactor(n_neighbors=3, contamination='auto')
lof.fit(X2)
sk_lof = -lof.negative_outlier_factor_
print_ranked_scores(X2, sk_lof)
```

Rank	Point	Score
1.	[2. 2.]	1.8608
2.	[1.9 2.]	1.8293
3.	[0.2 0.5]	1.2154
4.	[1. 0.5]	1.2091
5.	[0. 0.]	1.0333
6.	[0. 1.]	1.0120
7.	[1. 0.]	0.9554
8.	[0.9 1.]	0.8891
9.	[0.5 0.5]	0.8272

4.

```
def do_nn_avg_scores(obs,n_neighbors=1):
    nn = neighbors.NearestNeighbors(n_neighbors=n_neighbors)
    nn.fit(obs)
    dists, idx = nn.kneighbors()
    scores = dists[:, -1]
    mean=ss.tmean(scores)
    return mean

do_nn_avg_scores(X2, n_neighbors=1)
```

0.3763261014206866

```
def do_nn_harm_scores(obs,n_neighbors=1):
    nn = neighbors.NearestNeighbors(n_neighbors=n_neighbors)
    nn.fit(obs)
    dists, idx = nn.kneighbors()
    scores = dists[:, -1]
    mean=ss.hmean(scores)
    return mean

do_nn_harm_scores(X2, n_neighbors=1)
```

0.24764915747938757

```
def do_nn_avg_scores(obs,n_neighbors=1):
    nn = neighbors.NearestNeighbors(n_neighbors=n_neighbors)
    nn.fit(obs)
    dists, idx = nn.kneighbors()
    scores = dists[:, -1]
    mean=ss.tmean(scores)
    return mean

do_nn_avg_scores(X2, n_neighbors=3)
```

0.9938247446568358

```
def do_nn_harm_scores(obs,n_neighbors=1):
    nn = neighbors.NearestNeighbors(n_neighbors=n_neighbors)
    nn.fit(obs)
    dists, idx = nn.kneighbors()
    scores = dists[:, -1]
    mean=ss.hmean(scores)
    return mean

do_nn_harm_scores(X2, n_neighbors=3)
```

0.8302736306831558

5.

```
[129] def do_lof_outlier_method(obs, n_neighbors=3):
    neigh = neighbors.NearestNeighbors(n_neighbors=2).fit(obs)

    #Return indices of and distances to the neighbors of each point
    topk_dist, my_kneigh = neigh.kneighbors()

    # Create list of distances of furthest (kth) neighbor
    k_dist = topk_dist[:, -1]

    # Reachability distance: maximum of true distance between query neighbor and query point
    # and distance to kth nearest neighbor of query neighbor
    reach = np.maximum(topk_dist, k_dist[my_kneigh])

    # Local reachability density is reciprocal of average reachability distance
    lrd = 1.0 / np.mean(reach, axis=1)

    # Local outlier factor is given by
    # average local density of neighbors / local density of query point
    lrd_ratios = lrd[my_kneigh] / lrd[:, np.newaxis]
    lof = np.mean(lrd_ratios, axis=1)

    return lof
```

do_lof_outlier_method(X2,3)

```
array([1.0302, 1.1157, 1.0812, 1.1157, 0.8603, 1.0657, 1.0812, 1.761 ,
       1.761 ])
```